Karol Gąsior
Indeks 280913

Mateusz Kunicki
Indeks 272468

Grupa G2 (pon. 13:40)

Wrocław, 09-11-2026 r.

Opis założeń projektu

1 Temat i cel projektu

Temat: Mechanizmy zarządzania wątkami programowymi w zadaniu wykrywania krawędzi na obrazach cyfrowych z wykorzystaniem operatorów gradientowych (Sobela i pokrewnych).

Cel projektu: projekt i implementacja programu wykrywającego krawędzie na obrazie cyfrowym w pięciu wersjach oraz zmierzenie, ile czasu przetwarzania oszczędza każdy rodzaj przyspieszenia. Wersja W1 jest prostą implementacją sekwencyjną i stanowi punkt odniesienia. Każda z wersji W2–W5 bierze kod W1 i wprowadza do niego *jeden* rodzaj przyspieszenia. Wersja W2 nie korzysta z równoległości: zmienia algorytm i sposób korzystania z pamięci, aby sprawdzić, ile daje sama optymalizacja kodu na jednym rdzeniu. Wersje W3–W5 odpowiadają kolejnym poziomom przetwarzania równoległego [4]: równoległości na poziomie instrukcji procesora, czyli SIMD (W3), równoległości wątków na wielu rdzeniach procesora (W4) i masowej równoległości danych na setkach rdzeni karty graficznej (W5). Wersje nie będą łączone ze sobą. Każda zostanie porównana z W1, dzięki czemu zmierzony zysk pokaże wkład jednego mechanizmu, a nie sumę kilku.

W ujęciu przedmiotu jest to problem zarządzania zasobami obliczeniowymi komputera: rdzeniami procesora i ich wątkami sprzętowymi, pamięcią podręczną i operacyjną oraz kartą graficzną. Program musi rozdzielić pracę (wiersze obrazu albo piksele) pomiędzy te zasoby, zsynchronizować wątki i ograniczyć przesyłanie danych tak, aby czas przetwarzania był jak najkrótszy, a wynik pozostał poprawny. Najwięcej uwagi zostanie poświęcone wersji W4, w której te decyzje podejmuje własny mechanizm zarządzania wątkami (pula wątków i scheduler).

Zakres projektu obejmuje:

- maski Sobela 3×3, 5×5 i 7×7, Prewitta, Scharra i Robertsa;
- obrazy od 0,04 do 59 Mpix (zdjęcia i obrazy syntetyczne);
- w wersji wielowątkowej W4 – mechanizmy zarządzania wątkami: liczbę wątków od 1 do 24 (oraz 32 i 48), cztery strategie przydziału pracy, rozmiar przydzielanego fragmentu, sposób synchronizacji wątków przy wyznaczaniu wartości maksymalnej, pulę wątków w porównaniu z tworzeniem wątków przy każdym etapie i przypisanie wątków do rdzeni procesora;
- w wersji GPU W5 – liczbę wątków uruchamianych na karcie graficznej, rozmiar grupy roboczej i koszt przesyłania danych między pamięcią komputera a kartą;
- dwa pomiary każdej wersji: samego algorytmu oraz całego programu z wczytaniem i zapisem obrazu;

- ocenę, czy większa maska daje lepszy wynik wykrywania krawędzi, mierzoną na obrazach o znanym położeniu krawędzi z dodanym szumem;
- propozycje dalszych przyspieszeń przygotowane z pomocą modelu sztucznej inteligencji.

2 Sformułowanie problemu

Problem będzie rozwiązywany przy następujących założeniach:

- obraz wejściowy jest wczytywany z pliku (JPG, PNG, GIF) i konwertowany do skali szarości o 8 bitach na piksel albo generowany syntetycznie w pamięci;
- cały obraz mieści się w pamięci operacyjnej;
- piksele leżące poza brzegiem obrazu zastępuje się najbliższym pikselem brzegowym (powielenie krawędzi);
- wynikiem jest obraz 8-bitowy o tych samych wymiarach co wejście;
- najmniejszą jednostką podziału pracy pomiędzy wątki procesora jest wiersz obrazu, a pomiędzy wątki karty graficznej – piksel;
- wszystkie wersje muszą dawać wynik identyczny co do bitu z wynikiem wersji W1; jedynym wyjątkiem jest pomocniczy wariant GPU liczący zawsze w pojedynczej precyzji, dla którego dopuszcza się różnicę o 1 poziom szarości.

Problem obliczeniowy [2, 3]: dla obrazu I o wymiarach $W \times H$ i pary masek K_x, K_y o rozmiarze $k \times k$ wyznaczyć dla każdego piksela moduł gradientu

$$G(x,y) = \sqrt{G_x(x,y)^2 + G_y(x,y)^2}, \qquad G_x = K_x * I, \qquad G_y = K_y * I, \tag{1}$$

gdzie $*$ oznacza dwuwymiarowy splot dyskretny w postaci przyjętej w przetwarzaniu obrazów [2]

$$(K * I)(x,y) = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} K(i,j)\; I(x + j - \lfloor k/2 \rfloor,\; y + i - \lfloor k/2 \rfloor)\,, \tag{2}$$

Formalnie wzór (2) opisuje korelację, ponieważ maska nie jest odwrócona. Dla masek gradientowych odwrócenie zmienia tylko znak G_x i G_y, a nie moduł G, więc wynik jest ten sam. Należy następnie przeskalować wynik do zakresu 0–255: $O(x,y) = \operatorname{round}\left(255 \cdot G(x,y) / \max G\right)$. Skalowanie względem największej wartości w obrazie wymaga znajomości $\max G$, czyli zakończenia obliczeń dla całego obrazu przed zapisaniem pierwszego piksela wyniku. Jako warianty pomocnicze zostaną zbadane skalowanie stałe (względem teoretycznego maksimum maski) i obcięcie wartości większych od 255, które tego wymagania nie mają.

Parametrami problemu są:

- dane wejściowe: wymiary obrazu $W \times H$, maska (rodzaj i rozmiar k), sposób skalowania wyniku;
- zasoby sprzętowe: liczba rdzeni procesora i wątków sprzętowych, pojemność pamięci podręcznej, liczba rdzeni karty graficznej;

- zmienne decyzyjne, czyli sposób zarządzania zasobami: wersja programu, liczba wątków N, strategia przydziału wierszy do wątków i rozmiar fragmentu c, sposób synchronizacji przy wyznaczaniu maksimum, pula wątków albo nowe wątki, przypisanie wątków do rdzeni, a dla karty graficznej – rozmiar grupy roboczej, liczba pikseli na wątek i precyzja obliczeń.

Ograniczeniami są: wynik identyczny co do bitu z W1 (z wyjątkiem wariantu GPU w pojedynczej precyzji, dla którego dopuszcza się różnicę o 1 poziom szarości), liczba wątków sprzętowych procesora (24) i rdzeni karty (768), pojemność pamięci karty (4 GB) oraz bariera wynikająca ze skalowania względem maksimum: drugi etap nie może się rozpocząć, zanim pierwszy nie zakończy się dla całego obrazu.

Problem decyzyjny: czy dla danego obrazu, maski i sprzętu istnieje wersja programu i jej konfiguracja (liczba wątków, strategia przydziału pracy, rozmiar fragmentu, rozmiar grupy roboczej GPU), dla której czas przetwarzania nie przekracza zadanej wartości $T_{\max}$?

Problem optymalizacyjny [4]: dla każdej wersji dobrać wartości zmiennych decyzyjnych tak, aby zapewnić optymalne wartości następujących kryteriów:

- czas zegarowy przetwarzania obrazu – minimum;
- przyspieszenie względem wersji W1, $S = T_{W1}/T_{Wi}$ – maksimum;
- dla wersji wielowątkowej: przyspieszenie $S(N) = T(1)/T(N)$ i efektywność $E(N) = S(N)/N$ – maksimum; nierównowaga obciążenia wątków (różnica czasów zakończenia pracy) – minimum;

Osobnym kryterium jest jakość wykrywania krawędzi. Zależy ona od wyboru maski, a nie od wersji programu, ponieważ wszystkie wersje mają dawać ten sam wynik. Dlatego zostanie zbadana oddzielnie (punkt 6).

3 Analiza złożoności obliczeniowej problemu

Samo wyznaczenie modułu gradientu jest problemem obliczeniowym rozwiązywalnym w czasie wielomianowym, czyli z klasy P (ściślej FP, ponieważ wynikiem jest obraz, a nie odpowiedź tak/nie): istnieje algorytm deterministyczny o złożoności wielomianowej, który oblicza wynik dokładnie, bez przeszukiwania przestrzeni rozwiązań. Splot dwuwymiarowy wymaga dla każdego piksela k^2 mnożeń z dodawaniem dla G_x i tyle samo dla G_y, czyli $2k^2$ operacji. Całe zadanie ma więc złożoność $O(WHk^2)$, liniową względem liczby pikseli. Maski Sobela, Prewitta i Scharra są rozdzielne: każdą z nich można zapisać jako iloczyn wektora wygładzającego s i różnicującego d, np. dla Sobela 3×3:

$$K_x = s^T d = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}, \qquad K_y = d^T s. \tag{3}$$

Splot z maską rozdzielną można wtedy policzyć jako dwa sploty jednowymiarowe: poziomy (dla każdego wiersza obrazu wektorami s i d) i pionowy (na wynikach przebiegu poziomego). Wymaga to $4k$ zamiast $2k^2$ operacji na piksel, co daje złożoność $O(WHk)$: dla $k = 3$ jest to 12 zamiast 18 operacji, a dla $k = 7$ – 28 zamiast 98. Maska Robertsa nie jest rozdzielna. Skalowanie względem maksimum wymaga dodatkowego przebiegu $O(WH)$ i redukcji maksimum. Złożoność pamięciowa wynosi $O(WH)$: obraz wejściowy (1 B/piksel), bufor modułu gradientu (4 B/piksel) i wynik (1 B/piksel), łącznie około 6 B/piksel.

Wartość każdego piksela wyniku zależy wyłącznie od jego otoczenia w obrazie wejściowym, więc zadanie jest łatwo zrównoleglane i teoretycznie pozwala na przyspieszenie liniowe względem liczby jednostek obliczeniowych. Przyspieszenie ogranicza jednak część sekwencyjna programu, zgodnie z prawem Amdahla [10, 4]:

$$S(N) \le \frac{1}{s + \dfrac{1-s}{N}}, \qquad \lim_{N\to\infty} S(N) = \frac{1}{s}, \tag{4}$$

gdzie s oznacza udział części sekwencyjnej w czasie wykonania na jednej jednostce. W notacji wykładu [1] przyspieszenie oznacza się A_p, a liczbę procesorów p; w tym dokumencie odpowiadają im $S(N)$ i liczba wątków N. W tym zadaniu część sekwencyjną tworzą: wczytanie i dekodowanie pliku obrazu, zapis wyniku, przydział buforów, redukcja maksimum i bariera między etapami, a w wersji GPU także przesłanie obrazu na kartę i wyniku z powrotem. Drugim ograniczeniem jest przepustowość pamięci. Splot 3×3 wykonuje około 18 operacji na piksel, a jednocześnie przesyła z pamięci i do pamięci około 6 B na piksel, czyli zaledwie około 3 operacje na bajt danych. Po przyspieszeniu obliczeń czas może więc zależeć bardziej od szybkości odczytu i zapisu pamięci niż od mocy obliczeniowej, szczególnie gdy obraz nie mieści się w pamięci podręcznej.

Problem zarządzania, czyli wybór konfiguracji, ma inny charakter niż samo obliczenie. Problem decyzyjny i optymalizacyjny z punktu 2 dotyczą skończonego, niewielkiego zbioru konfiguracji (kilka wersji, do 48 wątków, 4 strategie przydziału, kilka rozmiarów fragmentu i grupy roboczej), więc można je rozwiązać dokładnie przeglądem zupełnym, w czasie wielomianowym względem liczby wartości każdego parametru. Trudność nie leży w złożoności obliczeniowej, ale w tym, że wartość kryterium (czas) nie wynika ze wzoru i trzeba ją zmierzyć dla każdej konfiguracji z powtórzeniami. Podproblem przydziału wierszy do wątków jest problemem szeregowania zadań na N identycznych procesorach z kryterium długości uszeregowania ($P \,||\, C_{\max}$) [5]. Gdy wszystkie wiersze wymagają tyle samo pracy, optymalny jest równy podział (po $\lceil H/N \rceil$ wierszy na wątek), co realizuje strategia *static*. W rzeczywistości czas wiersza zależy od chwilowej dostępności rdzenia (inne procesy, SMT, przerwania), a problem $P \,||\, C_{\max}$ z różnymi czasami zadań jest NP-trudny. Dlatego strategie *dynamic* i *guided* stosują szeregowanie listowe: zadanie trafia do pierwszego wolnego wątku. Jest to heurystyka, która daje uszeregowanie co najwyżej $2 - 1/N$ razy dłuższe od optymalnego [5].

Liczbę operacji i sposób zrównoleglenia w poszczególnych wersjach zestawiono w tabeli 1.

4 Metoda i algorytmy rozwiązywania problemu

4.1 Wersja W1 – sekwencyjna (punkt odniesienia)

Wersja W1 realizuje wzór (1) wprost, w jednym wątku. Dla każdego piksela (x, y) obrazu program:

1. przechodzi po wszystkich $k \times k$ polach maski; dla każdego pola wyznacza współrzędne piksela źródłowego i obcina je do zakresu obrazu funkcją `std::clamp` (także wtedy, gdy piksel leży daleko od brzegu i obcięcie nie jest potrzebne);
2. odczytuje współczynniki K_x i K_y z tablicy dynamicznej (`std::vector`), której rozmiar jest znany dopiero w czasie wykonania, i sumuje iloczyny w zmiennych całkowitych G_x i G_y;

Tabela 1: Złożoność i równoległość wersji programu (obraz $W \times H$, maska $k \times k$, N wątków procesora, P rdzeni karty graficznej).

Wersja	Operacje na piksel	Równoległość	Złożoność i oczekiwany czas
W1	$2k^2$ mnożeń, $2k^2$ obcięć współrzędnych	brak	$O(WHk^2)$
W2	$4k$ mnożeń (rozdzielne), bez obcięć we wnętrzu	brak	$O(WHk)$
W3	$2k^2$ mnożeń wykonywanych po 8 naraz	8 pikseli na instrukcję (AVX2)	$O(WHk^2)$, czas do 8 razy krótszy
W4	$2k^2$ mnożeń, jak W1	N wątków na rdzeniach CPU	$O(WHk^2)$ pracy, czas do N razy krótszy, ograniczony częścią sekwencyjną
W5	$2k^2$ mnożeń, jak W1	do kilku milionów wątków GPU na 768 rdzeniach	$O(WHk^2)$ pracy rozłożonej na P rdzeni, plus przesył danych

3. oblicza moduł $\sqrt{G_x^2 + G_y^2}$ w arytmetyce podwójnej precyzji i zapisuje go jako liczbę `float` do bufora modułu, aktualizując przy tym bieżące maksimum.

Po przetworzeniu całego obrazu drugi przebieg mnoży każdą wartość bufora przez $255/\max G$, zaokrągla ją i zapisuje jako bajt wyniku. Bufor modułu i bufor wyniku są przydzielane i zerowane przy każdym wywołaniu. Wersja ta jest najprostszą poprawną implementacją i jej wynik jest wzorcem, z którym porównywane są wszystkie pozostałe wersje.

Moduł liczony jest w podwójnej precyzji celowo: iloczyny G_x^2 i G_y^2 są wtedy dokładne, a pierwiastek poprawnie zaokrąglony, więc wynik nie zależy od kolejności działań ani od tego, czy kompilator połączy mnożenie z dodawaniem w jedną instrukcję. Pozwala to wymagać od każdej wersji wyniku identycznego co do bitu.

4.2 Wersja W2 – optymalizacja algorytmu i dostępu do pamięci (1 rdzeń)

Wersja W2 działa na jednym rdzeniu i nie używa instrukcji wektorowych w sposób jawny. Wprowadza cztery zmiany w samym algorytmie i organizacji danych:

1. **Obsługa brzegu poza główną pętlą.** Obcinanie współrzędnych jest potrzebne tylko w pasach szerokości $\lfloor k/2 \rfloor$ przy brzegach obrazu. Wnętrze obrazu jest przetwarzane osobną pętlą bez `std::clamp`, a wskaźniki do k wierszy źródłowych są wyznaczane raz na wiersz wyniku zamiast przy każdym odczycie. Znikają rozgałęzienia i przeliczanie indeksów w najczęściej wykonywanym fragmencie kodu.

2. **Maska jako stała czasu kompilacji.** Rozmiar maski i jej współczynniki są parametrami szablonu C++ (`constexpr`). Kompilator rozwija pętle po masce, pomija mnożenia przez zero (dla Sobela 3×3 trzy z dziewięciu współczynników każdej maski są zerowe) i zamienia mnożenia przez małe stałe na tańsze operacje.

3. **Splot rozdzielny.** Dla masek rozdzielnych każdy wiersz obrazu jest najpierw splatany poziomo z wektorami s i d, a wyniki k ostatnich wierszy są przechowywane w buforze

pierścieniowym. Splot pionowy korzysta z tego bufora. Każdy wiersz źródłowy jest przetwarzany poziomo tylko raz, a liczba operacji spada z $2k^2$ do $4k$ na piksel. Bufor k wierszy mieści się w pamięci podręcznej L1/L2 procesora.

4. **Mniejsze typy i bufory wielokrotnego użytku.** Wyniki przebiegu poziomego mieszczą się w 16 bitach (dla Sobela 7×7 co najwyżej $255 \cdot 64 = 16\,320$), więc bufor pierścieniowy przechowuje liczby `int16` zamiast `int32`, co zmniejsza o połowę ilość przesyłanych danych. Bufory są wyrównane do 64 B (rozmiar linii pamięci podręcznej) i przydzielane raz, a nie przy każdym wywołaniu, co usuwa koszt przydziału i pierwszego zapisu stron pamięci.

Ponieważ zysk ze splotu rozdzielnego rośnie z rozmiarem maski, oczekuje się, że W2 będzie zyskiwać względem W1 tym więcej, im większe jest k.

4.3 Wersja W3 – wektoryzacja SIMD (1 rdzeń)

Wersja W3 zachowuje algorytm W1: splot dwuwymiarowy $k \times k$, współczynniki maski odczytywane z tablicy w czasie wykonania (bez pomijania zer i bez rozdzielności), ten sam sposób liczenia modułu, przydział buforów przy każdym wywołaniu. Zmianą jest wykonywanie obliczeń dla 8 pikseli jednocześnie instrukcjami wektorowymi AVX2 [12], czyli równoległość na poziomie instrukcji procesora:

- dla każdego pola maski 8 sąsiednich pikseli (8 bajtów) jest wczytywanych jedną instrukcją i rozszerzanych do ośmiu liczb 32-bitowych w 256-bitowym rejestrze (`_mm256_cvtepu8_epi32`);
- współczynnik maski jest powielany do wszystkich 8 pozycji rejestru, a mnożenie i dodawanie (`_mm256_mullo_epi32`, `_mm256_add_epi32`) wykonują się dla 8 pikseli w jednej instrukcji;
- moduł jest liczony wektorowo: pierwiastkiem z 8 liczb pojedynczej precyzji (`_mm256_sqrt_ps`), gdy $G_x^2 + G_y^2 < 2^{24}$ i wynik jest dokładny, a w przeciwnym razie 2 razy po 4 liczby podwójnej precyzji (`_mm256_sqrt_pd`);
- drugi przebieg (skalowanie do 0–255) również przetwarza 8 pikseli na instrukcję.

Instrukcji wektorowej nie da się zastosować do pikseli brzegowych, dla których każdy z 8 pikseli wymagałby innego obcięcia współrzędnych. Dlatego kolumny brzegowe są liczone kodem W1, a w ścieżce wektorowej obcinany jest tylko numer wiersza źródłowego (raz na wiersz maski). Ta zmiana nie jest celem W3, ale zostanie wymuszona przez wektoryzację. Ma jednak skutek uboczny: ścieżka wektorowa nie obcina współrzędnych kolumn we wnętrzu obrazu, więc część zysku W3 będzie pochodzić z usunięcia tych obcięć, czyli z optymalizacji, którą W2 wprowadza osobno. Zostanie to uwzględnione przy interpretacji wyników. Teoretyczna granica przyspieszenia samej wektoryzacji wynosi 8. W praktyce zysk będzie mniejszy z powodu kosztu rozszerzania bajtów do 32 bitów, części skalarnej przy brzegach i przepustowości pamięci.

4.4 Wersja W4 – wątki na wielu rdzeniach procesora

Wersja W4 wykonuje kod W1 bez zmian (to samo jądro obliczeniowe), ale dzieli obraz pomiędzy N wątków. Zastosowana zostanie dekompozycja danych: obraz jest dzielony na fragmenty złożone z kolejnych wierszy, a każdy wątek zapisuje wyniki do rozłącznego

obszaru pamięci, więc same obliczenia nie wymagają blokad. Przetwarzanie przebiega w dwóch etapach rozdzielonych barierą:

1. splot i wyznaczenie lokalnego maksimum przez każdy wątek;
2. redukcja lokalnych maksimów do maksimum globalnego, a następnie równoległe skalowanie wyniku.

Wątki są tworzone raz i utrzymywane w puli (*thread pool*): między etapami czekają uśpione na zmiennej warunkowej (`std::condition_variable`) i są budzone, gdy pojawia się praca [6, 8]. Przydziałem fragmentów zarządza scheduler oferujący cztery strategie wzorowane na klauzuli `schedule` standardu OpenMP [9]:

- *static* – obraz dzielony na N równych bloków, po jednym na wątek; brak narzutu, ale wątek opóźniony przez system wydłuża cały etap;
- *cyclic* – fragmenty po c wierszy przydzielane kolejnym wątkom po kolei (fragment i dla wątku $i \bmod N$);
- *dynamic* – wolny wątek pobiera następny fragment ze wspólnego licznika atomowego; dobre równoważenie obciążenia kosztem operacji atomowej na każdy fragment;
- *guided* – jak *dynamic*, ale rozmiar fragmentu maleje wraz z ilością pozostałej pracy (duże fragmenty na początku, małe na końcu).

W wersji W4 zostaną zbadane następujące mechanizmy zarządzania wątkami:

- liczba wątków N od 1 do 24 oraz 32 i 48 (więcej wątków niż rdzeni logicznych), z uwzględnieniem topologii procesora: 12 rdzeni fizycznych po 2 wątki sprzętowe (SMT), podzielonych na 2 bloki (CCD) po 6 rdzeni z osobną pamięcią podręczną L3;
- strategia przydziału i rozmiar fragmentu c od 1 do 256 wierszy;
- sposób wyznaczania maksimum globalnego: lokalne maksimum każdego wątku i redukcja po barierze; porównanie każdego piksela ze wspólną zmienną atomową (`std::atomic`, zapis operacją porównaj-i-zamień); aktualizacja wspólnej zmiennej pod blokadą `std::mutex` dla każdego piksela; zapis bieżącego maksimum do własnej komórki wątku przy każdym pikselu – z komórkami w osobnych liniach pamięci podręcznej albo w jednej linii, co wywołuje fałszywe współdzielenie (*false sharing*) [11];
- pula wątków w porównaniu z tworzeniem nowych wątków (`std::thread`) w każdym etapie;
- przypisanie wątków do rdzeni logicznych (`pthread_setaffinity_np`): bez przypisania (decyduje system), najpierw jeden blok rdzeni, naprzemiennie oba bloki, tylko jeden wątek na rdzeń fizyczny;
- koszt bariery wynikającej ze skalowania względem maksimum, w porównaniu ze skalowaniem stałym, które wymaga tylko jednego etapu.

Oczekuje się, że przyspieszenie W4 będzie ograniczone przez część sekwencyjną kodu W1 (przydział i zerowanie buforów wykonywane jednym wątkiem przed podziałem pracy), barierę między etapami i przepustowość pamięci.

4.5 Wersja W5 – karta graficzna (OpenCL)

Wersja W5 przenosi algorytm W1 bez zmian na kartę graficzną NVIDIA GeForce GTX 1050 Ti (768 rdzeni CUDA w 6 multiprocesorach) przy użyciu standardu OpenCL [13]. W podstawowej konfiguracji każdy wątek karty (*work-item*) oblicza jeden piksel: czyta $k \times k$ pikseli z pamięci globalnej karty, mnoży je przez współczynniki maski z pamięci stałej i oblicza moduł. Przebieg programu:

1. przesłanie obrazu z pamięci komputera do pamięci karty przez magistralę PCIe;
2. kernel splotu; w każdej grupie roboczej wątki wyznaczają wspólne maksimum w pamięci lokalnej, a jeden wątek grupy aktualizuje maksimum globalne operacją atomową;
3. odczyt maksimum przez procesor i obliczenie skali;
4. kernel skalowania do 0–255;
5. przesłanie wyniku z powrotem do pamięci komputera.

Czas każdego etapu zostanie zmierzony osobno (zdarzenia profilujące OpenCL). Inicjalizacja (wybór urządzenia, kompilacja kernela) i przydział buforów na karcie odbędą się poza pomiarem.

Liczba rdzeni karty jest stała, ale programista wybiera liczbę wątków uruchamianych na karcie oraz rozmiar grupy roboczej (wątków współdzielących pamięć lokalną i wykonywanych na jednym multiprocesorze). Zostanie zbadany wpływ rozmiaru grupy roboczej (8×8, 16×16, 32×8, 32×32) i liczby pikseli liczonych przez jeden wątek (1–64, czyli od ok. 8,3 mln do ok. 140 tys. wątków dla obrazu 4K). Karta tej klasy liczy w podwójnej precyzji około 32 razy wolniej niż w pojedynczej, dlatego moduł będzie liczony w podwójnej precyzji tylko wtedy, gdy pojedyncza nie zapewnia wyniku dokładnego; dodatkowo zostanie zmierzony wariant zawsze w pojedynczej precyzji z oceną różnicy wyniku. Oczekuje się, że przesył danych przez PCIe będzie stanowił część sekwencyjną ograniczającą zysk dla małych obrazów.

4.6 Zestawienie wersji

Tabela 2 zestawia, co zmienia każda wersja względem W1 i jakie czynniki będą ograniczać jej przyspieszenie.

4.7 Metoda rozwiązania problemu optymalizacyjnego

Wartości zmiennych decyzyjnych zostaną dobrane przeglądem zupełnym siatki parametrów: dla każdej wersji zostaną zmierzone wszystkie zaplanowane kombinacje (np. dla W4 – liczba wątków $\times$ strategia przydziału $\times$ rozmiar fragmentu), a za rozwiązanie zostanie przyjęta konfiguracja o najmniejszej medianie czasu. Wyniki zostaną zweryfikowane modelem analitycznym tam, gdzie on istnieje: przyspieszenie W4 zostanie porównane z prawem Amdahla (4) dla dopasowanego udziału części sekwencyjnej, a czas W5 – z sumą zmierzonych czasów przesyłu i obliczeń. Parametry, których wpływ jest niezależny, będą badane osobno, przy pozostałych ustawionych na wartości domyślne, aby liczba pomiarów nie rosła wykładniczo z liczbą parametrów.

Wszystkie wersje mają dawać wynik dokładny, identyczny z W1. Jedynym rozwiązaniem przybliżonym jest wariant W5 liczący w pojedynczej precyzji. Jego dokładność zostanie oceniona eksperymentalnie: największą i średnią różnicą wyniku względem W1

Tabela 2: Wersje programu: rodzaj przyspieszenia i oczekiwane ograniczenia.

Wersja	Co zmienia względem W1	Poziom równoległości [4]	Oczekiwane ograniczenia
W1	–	brak	–
W2	brzegi poza pętlą, maska w czasie kompilacji, splot rozdzielny, `int16`, bufory wielokrotnego użytku	brak (1 rdzeń)	niewielki zysk rozdzielności dla $k = 3$; drugi przebieg skalowania
W3	8 pikseli na instrukcję (AVX2), skalowanie wektorowe	poziom instrukcji (SIMD)	rozszerzanie bajtów, brzegi skalarne, pamięć
W4	podział wierszy pomiędzy N wątków, pula, scheduler	poziom zadań (wątki, rdzenie)	część sekwencyjna, bariera, SMT, przepustowość pamięci
W5	obliczenia na karcie graficznej, 1 wątek na piksel	masowa równoległość danych	przesył PCIe, podwójna precyzja

na zestawie testowym. Przybliżony charakter ma też ocena jakości krawędzi przy progu dobranym optymalnie (punkt 6); wynik zostanie opisany jako górne oszacowanie jakości osiąganej w praktyce.

4.8 Propozycje dalszych przyspieszeń z pomocą AI

Wymagania projektu dopuszczają użycie systemów sztucznej inteligencji do podpowiedzi ulepszeń. Po wykonaniu pomiarów model językowy (Claude), mając dostęp do kodu i wyników, zaproponuje dalsze ulepszenia, których projekt jeszcze nie zawiera (np. wektorowe skalowanie we wszystkich wersjach, instrukcje AVX-512, duże strony pamięci, aktywne oczekiwanie wątków puli, równoległe kodowanie pliku wynikowego, nakładanie przesyłu i obliczeń na GPU). Propozycje zostaną opisane wraz z uzasadnieniem opartym na pomiarach i szacunkiem zysku, wyraźnie oznaczonym jako szacunek; ich wdrożenie i zmierzenie wykracza poza zakres projektu. W sprawozdaniu zostanie zaznaczone, że propozycje pochodzą od tego samego narzędzia, które wspierało implementację.

5 Metoda, technologie i narzędzia implementacji

Program zostanie napisany w języku C++ (standard C++17) z wykorzystaniem biblioteki STL, w szczególności mechanizmów wielowątkowości: `std::thread`, `std::mutex`, `std::condition_variable` i `std::atomic` [8, 16]. Wersja W3 użyje funkcji wewnętrznych kompilatora dla rozszerzenia AVX2 (`immintrin.h`) [12], kompilowanych tylko dla funkcji tej wersji, aby pozostała część programu działała na każdym procesorze x86-64. Wersja W5 użyje OpenCL 1.2 ze sterownikiem karty NVIDIA. Odczyt i zapis plików graficznych zrealizują jednoplikowe biblioteki `stb_image` i `stb_image_write` [18].

Logika obliczeń zostanie zamknięta w bibliotece `edge_core`, w której wszystkie wersje są wywoływane przez jedną funkcję. Gwarantuje to identyczne granice pomiaru czasu dla wszystkich wersji. Pomiary wykona konsolowa aplikacja `edge_bench` oparta na module Qt Core [17], uruchamiana z pliku opisującego eksperyment (JSON) i zapisująca surowe wyniki do plików CSV wraz z opisem warunków pomiaru (procesor, kompilator, flagi,

wersja kodu). Projekt będzie budowany za pomocą CMake i kompilatora GCC 13 w trybie Release (optymalizacja `-O3`). Skrypty w języku Python (biblioteki NumPy, pandas, Matplotlib, Pillow) uruchomią eksperymenty kolejno, przetworzą wyniki i wygenerują wykresy oraz tabele; dokumentacja zostanie przygotowana w systemie LaTeX.

Pomiary zostaną wykonane na komputerze z procesorem AMD Ryzen 9 9900X i kartą graficzną NVIDIA GeForce GTX 1050 Ti (4 GB) pod systemem Ubuntu 24.04. Procesor ma 12 rdzeni fizycznych i 24 wątki sprzętowe (SMT). Rdzenie są podzielone na 2 bloki (CCD) po 6 rdzeni, a każdy blok ma własną pamięć podręczną L3 o pojemności 32 MB. Każdy rdzeń ma 1 MB pamięci L2. Procesor obsługuje rozszerzenia AVX2 i AVX-512. Ta budowa ma znaczenie dla W4, ponieważ wątki z różnych bloków nie współdzielą pamięci L3, a dwa wątki sprzętowe jednego rdzenia współdzielą jego jednostki wykonawcze [7].

6 Sposób testowania i oceny jakości rozwiązań

Poprawność. Wynik każdej wersji zostanie porównany z wynikiem W1 dla tego samego obrazu, maski i sposobu skalowania. Zestaw testowy obejmie przypadki brzegowe: obraz 1×1, jedną kolumnę, jeden wiersz, obraz mniejszy od maski 7×7, mniej wierszy niż wątków i wymiary nieparzyste, a dla W4 i W5 – różne liczby wątków, strategie przydziału, rozmiary fragmentu i grupy roboczej. Wymagany jest wynik identyczny co do bitu; dla wariantu GPU w pojedynczej precyzji zostanie podana największa różnica.

Pomiary czasu. Dla każdej wersji zostaną wykonane dwa pomiary:

1. **sam algorytm** – od obrazu w pamięci do gotowego obrazu wynikowego; dla W5 łącznie z przesłaniem danych na kartę i z powrotem;
2. **cały program** – wczytanie i zdekodowanie pliku obrazu, algorytm, zapis wyniku do pliku.

Analiza skupi się na samym algorytmie, ale dla każdej wersji zostanie pokazany rozkład czasu całego programu na wczytanie obrazu, algorytm i zapis wyniku, ponieważ operacje plikowe są sekwencyjne. Zgodnie z prawem Amdahla (4) mogą one ograniczać zysk całego programu bardziej niż sam algorytm. Czas algorytmu zostanie dodatkowo rozbity na etapy (splot, skalowanie, a dla W5 przesył do karty, obliczenia i przesył z karty).

Każda konfiguracja zostanie wykonana najpierw 1–2 razy bez zapisu wyniku (rozgrzewka: pierwsze wywołanie przydziela pamięć i ładuje kod do pamięci podręcznej), a następnie 11–15 razy z zapisem czasu. Jako wartość zostanie przyjęta mediana, a jako rozrzut – zakres od minimum do maksimum, zgodnie z zaleceniami Hoeflera i Belli [15]. Mediana jest mniej wrażliwa niż średnia na pojedyncze pomiary wydłużone przez system operacyjny. Czas będzie mierzony zegarem monotonicznym `std::chrono::steady_clock`. Eksperymenty będą uruchamiane kolejno, nigdy dwa naraz, a każdy z nich rozpocznie się tylko wtedy, gdy inne procesy zajmują mniej niż 5 % czasu procesora. Regulator częstotliwości procesora zostanie ustawiony w tryb *performance*, czyli bez obniżania częstotliwości przy małym obciążeniu. Każde przyspieszenie zostanie podane razem z czasem bezwzględnym wersji bazowej.

Dane testowe. Zdjęcie NASA Blue Marble ($21\,600 \times 10\,800$ pikseli) przeskalowane do 14 rozmiarów od 256×144 do $10\,240 \times 5\,760$ (0,04–59 Mpix), obrazy o tej samej liczbie

pikseli i różnych proporcjach, zdjęcia z zestawu Kodak (768×512) oraz obrazy syntetyczne z losowymi figurami. Duże obrazy posłużą do pomiarów czasu, a małe zdjęcia – do sprawdzenia poprawności i do rysunków wyniku filtru.

Plan eksperymentów. Zaplanowano następujące eksperymenty:

- porównanie wersji W1–W5 dla obrazów HD, 4K i 8K oraz masek 3×3 i 7×7, osobno dla samego algorytmu i całego programu;
- wpływ rozmiaru i rodzaju maski na czas każdej wersji;
- dla W4: przyspieszenie $S(N)$ i efektywność $E(N)$ w funkcji liczby wątków z dopasowaniem prawa Amdahla, strategia przydziału i rozmiar fragmentu, sposób wyznaczania maksimum, pula w porównaniu z nowymi wątkami, przypisanie do rdzeni oraz przyspieszenie całego programu w funkcji liczby wątków;
- dla W5: rozkład czasu na przesył i obliczenia, liczba wątków i rozmiar grupy roboczej, precyzja obliczeń;
- dla wszystkich wersji: czas i przepustowość w funkcji rozmiaru obrazu oraz rozmiar obrazu, od którego dana wersja jest szybsza od W1.

Jakość wykrywania krawędzi. Zostanie sprawdzone, czy większa maska daje lepszy wynik. Na syntetycznym obrazie z figurami o stałej jasności, dla którego położenie krawędzi jest znane co do piksela, zostanie dodany szum gaussowski o odchyleniu standardowym σ od 0 do 40 poziomów szarości. Wynik każdej maski zostanie zamieniony na obraz binarny progowaniem (próg dobierany osobno dla każdej maski i poziomu szumu tak, aby dawał najlepszy wynik, co porównuje maski w ich najlepszym ustawieniu) i porównany z prawdziwymi krawędziami miarą Pratta [14]:

$$F = \frac{1}{\max(N_I, N_D)} \sum_{j=1}^{N_D} \frac{1}{1 + \alpha\, d_j^2}, \qquad \alpha = \tfrac{1}{9}, \tag{5}$$

gdzie N_I – liczba prawdziwych pikseli krawędzi, N_D – liczba wykrytych, d_j – odległość j-tego wykrytego piksela od najbliższej prawdziwej krawędzi. $F = 1$ oznacza wykrycie idealne; miara karze zarówno brakujące i fałszywe krawędzie, jak i krawędzie przesunięte lub pogrubione. Oczekuje się, że większa maska, która mocniej wygładza obraz, będzie odporniejsza na szum, ale da grubsze i gorzej położone krawędzie przy obrazie bez szumu.

Prezentacja wyników. Wyniki zostaną przedstawione na wykresach generowanych bezpośrednio z surowych danych: czas i przyspieszenie każdej wersji względem W1 (z czasem bezwzględnym W1), rozkład czasu na etapy dla każdej wersji, przyspieszenie $S(N)$ wersji W4 z linią idealną $S = N$ i krzywą prawa Amdahla, czas w funkcji rozmiaru obrazu, jakość wykrywania w funkcji szumu. Wnioski zostaną poparte liczbami i odniesione do mechanizmów: liczby operacji, szerokości wektora, części sekwencyjnej, przepustowości pamięci i topologii procesora.

Literatura

[1] Wójcik R., *Zarządzanie w systemach i sieciach komputerowych*, materiały do wykładu, Wykład 2: Zasoby i procesy w systemach i sieciach komputerowych, Politechnika Wrocławska, Wrocław, 2026.

[2] Gonzalez R. C., Woods R. E., *Digital Image Processing*, 4th ed., Pearson, New York, 2018.

[3] Tadeusiewicz R., Korohoda P., *Komputerowa analiza i przetwarzanie obrazów*, Wydawnictwo Fundacji Postępu Telekomunikacji, Kraków, 1997.

[4] Czech Z., *Wprowadzenie do obliczeń równoległych*, PWN, Warszawa, 2010.

[5] Błażewicz J., *Problemy optymalizacji kombinatorycznej*, PWN, Warszawa, 1996.

[6] Karbowski A., Niewiadomska-Szynkiewicz E. (red.), *Programowanie równoległe i rozproszone*, Oficyna Wydawnicza Politechniki Warszawskiej, Warszawa, 2009.

[7] Silberschatz A., Peterson J. L., Gagne G., *Podstawy systemów operacyjnych*, WNT, Warszawa, 2005.

[8] Williams A., *C++ Concurrency in Action*, 2nd ed., Manning, Shelter Island, 2019.

[9] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Version 5.2, 2021.

[10] Amdahl G. M., *Validity of the single processor approach to achieving large scale computing capabilities*, AFIPS Conference Proceedings, vol. 30, 1967.

[11] Drepper U., *What Every Programmer Should Know About Memory*, Red Hat, 2007.

[12] Intel Intrinsics Guide, `https://www.intel.com/content/www/us/en/docs/intrinsics-guide/`, 09.11.2026.

[13] Khronos Group, *The OpenCL Specification*, Version 1.2, 2012.

[14] Abdou I. E., Pratt W. K., *Quantitative design and evaluation of enhancement/thresholding edge detectors*, Proceedings of the IEEE, vol. 67, no. 5, 1979, s. 753–763.

[15] Hoefler T., Belli R., *Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses when Reporting Performance Results*, Proceedings of SC'15, ACM, 2015.

[16] `https://en.cppreference.com/w/cpp/thread`, 19.10.2026.

[17] Qt 6 Documentation, `https://doc.qt.io/qt-6/`, 09.11.2026.

[18] `https://github.com/nothings/stb`, 19.10.2026.